MECE E3028 Mechanical Engineering Laboratory II
Professor Bianca Howard
Spring 2026

Experiment 6: Solar Tracker

LABORATORY REPORT

Lab Group 5

Joseph Berbner

David Culcay

Kristina Isabella Dee

Zain Erakky

Columbia University
Department of Mechanical Engineering

May 11th, 2026

Contents

Abstract **2**

List of Figures **3**

List of Tables **4**

1 Introduction **5**

2 Theory and Methods **6**

2.1 Initial Design of Dual Axis Solar Tracker 6

2.1.1 Description of Design Inspiration 6

2.1.2 Overview of Initial Solar Tracking System Design 6

2.1.3 Initial Mechanical and Electrical System Description 7

2.1.4 Electrical Component Diagram and Description 7

2.1.5 Initial Control Logic Description 8

2.1.6 Initial System Operation 9

2.1.7 Initial Arduino Coding Decisions and Rationale 9

2.2 Revised Design of Dual Axis Solar Tracker 10

2.2.1 Overview of Design Changes 10

2.2.2 Revised Mechanical and Electrical System 11

2.2.3 Revised System Operation 12

2.2.4 Revised Arduino Coding Decisions and Rationale 13

3 Experimental Apparatus **14**

4 Results and Discussion **16**

5 Conclusion **18**

A Initial Arduino Control Code **19**

B Revised Arduino Control Code **23**

References **29**

Abstract

This experiment involved the design and construction of a dual-axis solar tracking system. The primary objective was to develop a compact and accurate device capable of maintaining proper orientation relative to a moving light source. The apparatus consisted of a solar array, four photoresistors for light sensing, a DC motor for altitude adjustment, a stepper motor for azimuth rotation, and an Arduino-based control system with a motor shield. The system operated using a scan-and-sweep control method, where the tracker evaluated light intensity across its range of motion and adjusted its position to maximize incident light on the solar array. The mechanical design allowed for approximately 180° of motion in both altitude and azimuth, enabling full coverage of the tested light source positions.

Performance testing demonstrated reliable operation across most configurations, with the system successfully aligning to the light source in eight out of nine cases. The maximum output voltage of 2.65 V was recorded at an azimuth of 90° and an altitude of 75°, indicating near-optimal alignment. Although the system performed well overall, minor issues were observed at extreme angles, particularly at 15° altitude and 135° azimuth, where reduced light intensity led to tracking inaccuracies, slight mechanical jitter, and lower voltage output. These limitations can be addressed in future iterations by incorporating a higher-torque motor to improve motion stability and by reinforcing structural connections to reduce vibration. Overall, the system successfully demonstrated effective dual-axis tracking and consistent performance under controlled testing conditions.

List of Figures

1 Initial solar tracker prototype 7
2 Solar tracker electrical circuit 8
3 Solar tracker design modifications 11
4 Final solar tracker design 12
5 Solar tracker apparatus 14

List of Tables

1 Kit components and usage 15
2 Tracker voltage results 16

1 Introduction

The objective of this experiment was to design, construct, and evaluate a dual-axis solar tracking system capable of maintaining optimal alignment with a moving light source. The system was required to adjust both altitude and azimuth angles in response to changes in light position while remaining compact, stable, and mechanically reliable. A dual-axis solar tracker is a system that continuously orients a solar panel so that its surface remains as close as possible to perpendicular to incoming light. This maximizes the amount of solar energy captured by the panel, as peak power output occurs when the panel is directly normal to the light source :contentReference[oaicite:0]index=0. The position of the light source is defined by two angles: the altitude angle, which describes the vertical position of the light, and the azimuth angle, which describes its horizontal direction. By tracking both angles, a dual-axis system can achieve higher efficiency compared to fixed or single-axis configurations.

Two design approaches were developed and tested throughout the experiment. The initial design (Design 1) utilized a continuous tracking control strategy, where the system actively followed the light source in real time based on sensor input. This approach prioritized immediate response but resulted in a heavier and more constrained structure, with increased sensitivity to noise and mechanical instability. The revised design (Design 2) focused on reducing structural weight and improving overall mobility. This design implemented a scan-and-sweep control method, in which the system first evaluated light intensity across its range of motion in both vertical and horizontal directions, and then aligned itself with the position corresponding to maximum detected illumination. This approach improved stability and reduced oscillatory behavior compared to continuous tracking. The final system successfully demonstrated dual-axis tracking capability, achieving accurate alignment across most tested configurations. The design evolution from continuous tracking to scan-based positioning resulted in improved performance, smoother motion, and more reliable operation during experimental testing.

2 Theory and Methods

2.1 Initial Design of Dual Axis Solar Tracker

2.1.1 Description of Design Inspiration

The initial design of the dual-axis solar tracking system was inspired by the implementation presented in [3], which demonstrated a functional Arduino-based solar tracker using a multi-sensor light detection system and servo motors for actuation. This reference provided a clear foundation for understanding both the control strategy and overall system layout, particularly the use of multiple photoresistors to determine the direction of maximum light intensity. While the reference design utilized servo motors for both axes of rotation, the present system required modification to accommodate a 5V stepper motor for azimuth motion and a 6V DC motor for altitude adjustment, both controlled through an Arduino motor shield. As a result, the control logic and hardware integration were adapted to ensure compatibility with the available components while maintaining the same underlying tracking principle.

The transition from servo-based actuation to motor shield-driven control introduced additional considerations in power management and motion control, which are discussed further in Section 2.1.4.

2.1.2 Overview of Initial Solar Tracking System Design

The initial solar tracking system design focused on developing a structurally robust dual-axis platform capable of supporting both vertical (altitude) and horizontal (azimuth) motion. As shown in Figure 1, the system consisted of a base platform that housed the azimuth rotation mechanism, with a rotating upper stage mounted on top to support the vertical tracking assembly. The lower base provided stability and acted as the primary support structure, while the elevated central column enabled rotation about the vertical axis. Mounted on this rotating platform was a secondary frame that supported the solar panel and vertical actuation system. This upper frame allowed for controlled tilting motion, enabling adjustment of the panel's altitude angle relative to the incoming light source. The design prioritized structural rigidity and alignment between the two axes of motion; however, the initial configuration resulted in a relatively heavy and bulky system. The elevated mounting structure and enclosed frame increased overall mass, which contributed to higher load demands on the motors and reduced responsiveness during operation. These limitations motivated further refinement of the design in subsequent iterations.

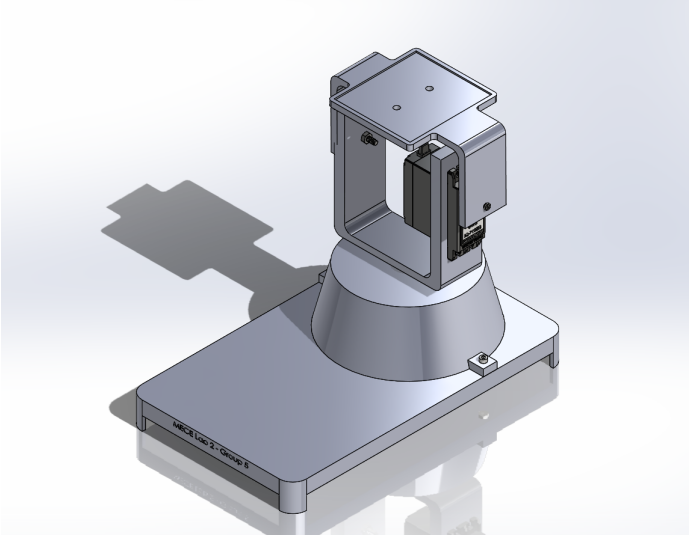


Figure 1: Initial dual-axis solar tracker prototype used during preliminary testing and subsystem validation.

2.1.3 Initial Mechanical and Electrical System Description

The mechanical and electrical layout of the initial system is illustrated in Figure 1. The design integrates both actuation and control components within a compact two-stage structure. The azimuth motion is achieved through a motor mounted beneath the cone-shaped base, which is not directly visible in the figure. This motor drives the rotation of the upper platform about the vertical axis. The altitude motion is controlled by a DC motor mounted on the vertical bracket of the upper assembly. This motor directly actuates the tilting motion of the solar panel, allowing adjustment of the panel angle relative to the incoming light source. The vertical bracket provides structural support for the motor while maintaining alignment of the rotation axis.

The electrical components, including the Arduino and breadboard, are positioned on the lower base platform. These components were secured using hot glue or a similar adhesive to prevent movement during operation while still allowing for quick adjustments during testing. This placement ensured a compact footprint and minimized interference with the rotating structure. While the system successfully integrated both mechanical and electrical components, the initial configuration resulted in a relatively dense layout, which contributed to increased weight and limited accessibility for wiring and adjustments.

2.1.4 Electrical Component Diagram and Description

The electrical layout of the dual-axis solar tracking system is shown in Figure 2. The system is centered around an Arduino microcontroller connected to an Adafruit motor shield, which provides the necessary power and control interface for both the stepper motor (azimuth motion) and the DC motor (altitude motion). The stepper motor is wired directly to the

motor shield to enable controlled rotational steps, while the DC motor is connected to allow continuous motion for vertical adjustment.

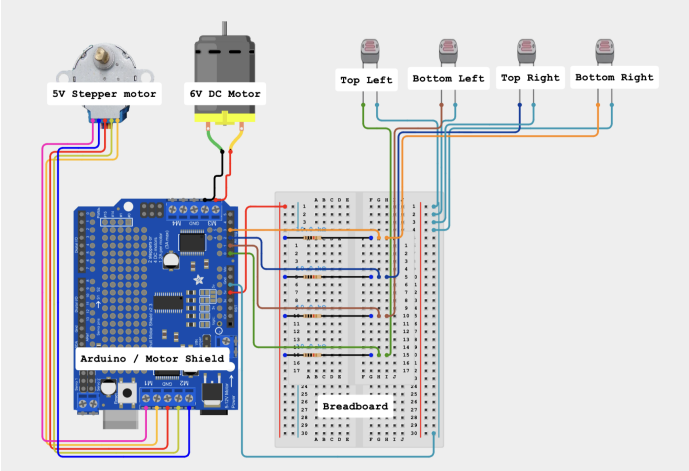


Figure 2: The electrical circuit layout for the dual-axis solar tracker, showing the Arduino, motor shield, LDR sensor connections, DC motor, stepper motor, and power connections. This was kept the same during the intial and final design.

Four photoresistors (LDRs) are arranged around the solar panel in a top-left, top-right, bottom-left, and bottom-right configuration. Each sensor is connected to the Arduino through a voltage divider circuit on the breadboard, allowing analog voltage readings that correspond to light intensity. These readings are used by the control algorithm to determine the direction of maximum illumination and adjust the panel orientation accordingly. The breadboard serves as the central hub for sensor integration and signal routing, while the motor shield manages higher power demands for the motors. Once this electrical configuration was established and verified, it remained consistent throughout both the initial and revised system designs, providing a reliable foundation for control and sensing. To ensure reliable operation during movement, careful attention was given to wire management. Wires were organized by color to simplify debugging and reduce confusion, and bundled using zip ties to prevent entanglement. Routing was planned to avoid interference with moving mechanical components, such as the rotating base and vertical bracket, ensuring that electrical connections remained secure throughout operation.

2.1.5 Initial Control Logic Description

The control logic of the solar tracking system refers to the set of programmed rules and decision-making processes used by the Arduino to interpret sensor inputs and determine the appropriate motor responses. In this experiment, the control logic governed how the system adjusted its orientation based on the measured light intensity from the photoresistors. The initial control strategy was based on real-time comparison of analog signals obtained from four photoresistors arranged around the solar panel. The sensors were grouped into pairs to

represent horizontal (left vs. right) and vertical (top vs. bottom) light intensity differences. By comparing the averaged values of these groups, the system determined the direction of higher illumination.

When a difference in light intensity was detected between the left and right sensor groups, the stepper motor was actuated to rotate the system in the azimuth direction toward the brighter side. Similarly, when a difference was detected between the top and bottom sensor groups, the DC motor was activated to adjust the altitude angle of the panel. This allowed the system to continuously respond to changes in light position and attempt to maintain alignment with the strongest light source. The initial implementation relied on simple threshold-based decision logic, where motors were triggered whenever the difference between sensor readings exceeded a predefined value. While this approach enabled basic tracking functionality, it also introduced limitations such as sensitivity to noise, oscillatory motion near equilibrium, and inconsistent directional response under low-light conditions.

2.1.6 Initial System Operation

The initial solar tracking system operated by continuously adjusting its orientation in response to light intensity measurements from the photoresistors. Upon activation, the system began monitoring the four sensors positioned at the corners of the rectangular solar panel (top-left, top-right, bottom-left, and bottom-right). These sensors provided real-time feedback on the direction of maximum incident light. Based on this feedback, the system actuated two motors to control its orientation. The stepper motor, located at the base, rotated the entire structure about the vertical axis to adjust the azimuth angle. Simultaneously, the DC motor mounted on the upper bracket controlled the altitude angle by tilting the solar panel assembly. This dual-axis motion allowed the system to align the panel toward the strongest light source. The control logic continuously compared the light intensity detected by the sensors and commanded incremental motor movements to reduce the difference between opposing sensor pairs. As a result, the system attempted to maintain a balanced condition in which all sensors received approximately equal light intensity, corresponding to optimal alignment.

2.1.7 Initial Arduino Coding Decisions and Rationale

The complete initial Arduino source code is provided in Appendix A. The initial implementation focused on establishing a reliable and responsive control framework that could integrate sensor input with motor actuation using the Adafruit Motor Shield. One key design decision was the use of a threshold-based control scheme to determine when motor movement should occur. Separate tolerance values were defined for horizontal and vertical motion, allowing the system to ignore small fluctuations in sensor readings and reduce unnecessary motor actuation. This approach provided a simple yet effective method for filtering noise while maintaining responsiveness to meaningful changes in light intensity.

Another important consideration was the implementation of bounded motion for both axes. Position limits were introduced for the stepper motor (azimuth) and DC motor (alti-

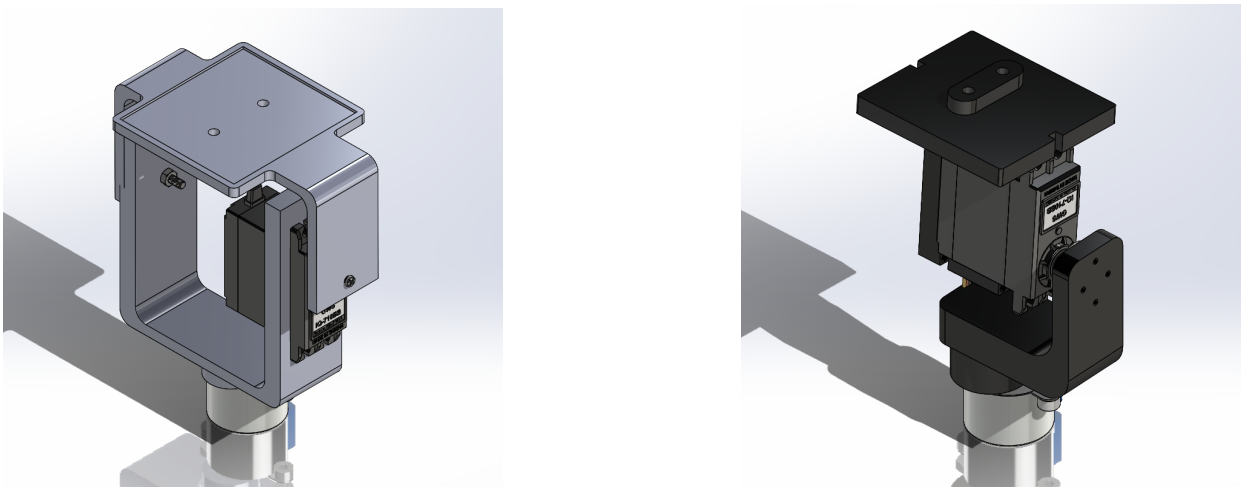
tude) to prevent over-rotation and protect the mechanical structure. This ensured that all motion remained within a safe operating range, improving system reliability during testing. The code also incorporated a startup calibration routine, which established baseline sensor readings under ambient lighting conditions. By subtracting these baseline values during operation, the system was able to better detect relative changes in light intensity rather than relying on absolute measurements. This improved tracking consistency across different lighting environments. Motor control parameters, such as step size, speed, and actuation duration, were selected to balance smooth motion with sufficient responsiveness. The stepper motor was configured for incremental movement to allow controlled azimuth adjustments, while the DC motor was actuated in short time intervals to approximate positional control in the vertical direction.

2.2 Revised Design of Dual Axis Solar Tracker

2.2.1 Overview of Design Changes

The primary modifications between the initial and revised solar tracker designs focused on reducing structural complexity, improving motor integration, and enhancing overall system mobility. As shown in Figure 3, the initial design (Figure 3a) featured a larger, enclosed frame that supported the vertical motion system. While this provided rigidity, it resulted in increased weight and limited accessibility to internal components. In the revised design (Figure 3b), the structure was simplified by reducing excess material and adopting a more compact mounting configuration for the stepper motor and vertical bracket. The stepper motor was repositioned and more directly integrated into the support structure, allowing for improved load transfer and more efficient rotation about the azimuth axis. Additionally, the revised design minimized the use of large enclosing geometries, which reduced overall mass and improved the responsiveness of the system.

These changes resulted in a lighter and more open design, which not only improved mechanical performance but also allowed easier access for wiring, adjustments, and maintenance. The revised configuration better supported the updated control strategy by enabling smoother motion and reducing the mechanical resistance observed in the initial prototype.



(a) Initial solar tracker design.

(b) Revised solar tracker design.

Figure 3: Comparison between the initial and revised dual-axis solar tracker configurations.

2.2.2 Revised Mechanical and Electrical System

The final revised mechanical and electrical configuration of the solar tracking system is shown in Figure 4. The updated design reflects improvements made to reduce structural weight, simplify motor integration, and enhance overall system performance during operation. The vertical and horizontal motion systems were more directly integrated, resulting in a more compact and stable structure compared to the initial design. The stepper motor was more effectively mounted within the base assembly to provide smoother and more reliable azimuth rotation, while the DC motor was securely attached to the vertical bracket to improve altitude control. The revised configuration reduced excess material and improved load distribution, which contributed to more responsive motion and reduced mechanical strain on the motors.

The electrical system layout remained largely consistent with the initial design; however, improvements were made in component organization and integration. Wiring was more carefully routed and secured to prevent interference with moving components, particularly around the rotating base and vertical bracket. Additionally, an external power supply was used to power the motors, ensuring sufficient current delivery and preventing over-reliance on the Arduino's onboard power, which improved overall system stability and performance. This resulted in a more robust and reliable system during testing. An additional modification was made to improve sensor performance. Small vertical divider fins were placed between the four photoresistors to minimize cross-light interference and ensure that each sensor more accurately detected light from its respective direction. This improved the system's ability to distinguish light gradients and enhanced overall tracking accuracy. A detailed view of the sensor arrangement can be seen in Figure 5 in the Experimental Apparatus section.

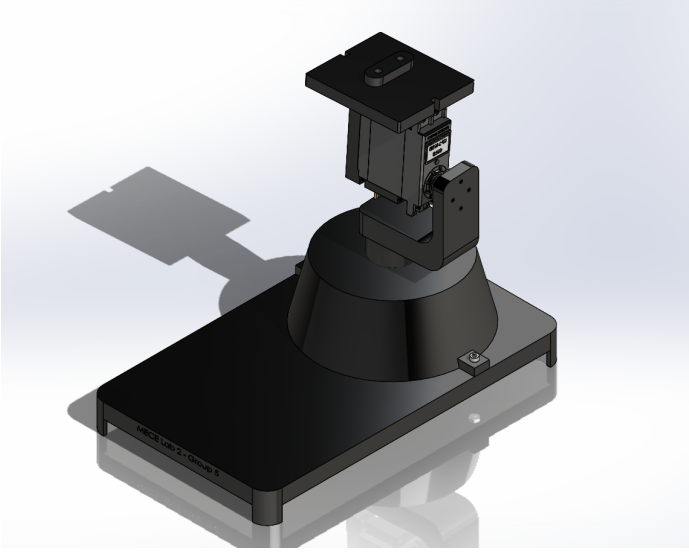


Figure 4: Final revised dual-axis solar tracker system used during experimental testing and voltage measurements.

2.2.3 Revised System Operation

The revised solar tracking system operated using a scan-and-sweep control strategy, which replaced the continuous tracking approach used in the initial design. Instead of constantly adjusting its position in response to small sensor differences, the system periodically scanned its range of motion to identify the direction of maximum light intensity and then moved to that position. For altitude (vertical motion), the system began at a defined home position and performed a sweep across its range of motion. The DC motor rotated the upper bracket approximately 90° in one direction and then 90° in the opposite direction while continuously monitoring light intensity from the photoresistors. During this sweep, the system recorded the position corresponding to the highest detected light intensity and subsequently returned to that position, effectively "locking" onto the optimal altitude angle.

A similar process was then applied for azimuth (horizontal motion). The stepper motor rotated the base through a comparable angular range, sweeping approximately 90° in each direction. As with the vertical sweep, the system tracked light intensity during motion and identified the azimuth position with the strongest illumination. The stepper motor then repositioned the system to this optimal angle. This sequential scan-and-lock approach reduced unnecessary motor actuation and eliminated oscillatory behavior near equilibrium conditions. By evaluating a range of positions before committing to movement, the system achieved more stable and accurate alignment with the light source compared to the initial continuous tracking method.

2.2.4 Revised Arduino Coding Decisions and Rationale

The complete revised Arduino source code is provided in Appendix B. The updated implementation focused on improving system stability, repeatability, and ease of experimental validation compared to the initial design. One of the primary coding decisions was to structure the program into clearly defined operational stages rather than continuous feedback loops. This allowed the system to complete distinct tracking actions in sequence, reducing the likelihood of conflicting motor commands and improving overall control clarity. By separating vertical and horizontal motion into independent routines, the system achieved more predictable and controlled behavior.

Another important modification was the refinement of motor control parameters to improve reliability. The stepper motor was operated at a reduced speed with smaller step increments to increase available torque and prevent stalling during rotation. Similarly, the DC motor was controlled using short, timed actuation intervals, which provided more consistent vertical positioning and reduced overshoot. To achieve the desired angular range of motion, additional tuning was required for both motors. For the DC motor, the duration of each actuation step was adjusted such that the total sweep corresponded to approximately 90° of rotation in each direction. For the stepper motor, the total number of steps and step size were calibrated to ensure that the system could rotate through a comparable angular range, allowing for consistent horizontal scanning. These adjustments ensured that both axes covered the intended field of view during the scan-and-sweep operation.

Sensor processing was also improved to enhance measurement consistency. Instead of relying on directional differences alone, the revised code utilized a combined brightness metric derived from all four photoresistors. This provided a more robust representation of overall light intensity and reduced sensitivity to localized fluctuations or uneven lighting conditions. Additionally, extensive serial monitoring was incorporated to aid in debugging and performance evaluation. Real-time outputs of sensor values, brightness trends, and motor position estimates allowed for easier tuning of system parameters and verification of correct operation during testing. The revised coding approach emphasized structured control, improved motor reliability, and enhanced data visibility. These decisions contributed to smoother system operation and more consistent tracking performance during experimental evaluation.

3 Experimental Apparatus

The components provided in the laboratory kit are summarized in Table 1. Each component is listed along with its role in the solar tracking system and whether it was incorporated into the final design. Although all components were initially considered during the design process, only those required for reliable dual-axis tracking were selected. The final system primarily utilized the Arduino Uno, Adafruit Motor Shield, DC motor, stepper motor, photoresistors, and supporting electrical connections. Components such as the potentiometer and limit switch were not included in the final implementation, as the chosen control strategy relied on light-based feedback and software-defined motion limits rather than direct position sensing or mechanical constraints.

The mechanical components of the solar tracker were designed in SolidWorks, with careful consideration given to spatial layout, weight distribution, and overall system size to ensure stable operation and proper alignment of the tracking axes. The design included mounting structures for the motors, support for the solar array, and attachment points to securely integrate the rotating components. These parts were fabricated using PLA through 3D printing, providing a lightweight and easily manufacturable solution suitable for rapid prototyping. Following fabrication, the components were assembled using screws to provide structural integrity and maintain alignment between the vertical and horizontal axes of rotation. Additional reinforcement and positioning of electrical components, such as the breadboard and wiring, were achieved using hot glue. This allowed for quick adjustments during testing while ensuring that components remained fixed during operation. Finally, double header pins were soldered onto the motor shield to allow for secure and convenient electrical connections between the Arduino and the circuit wiring.

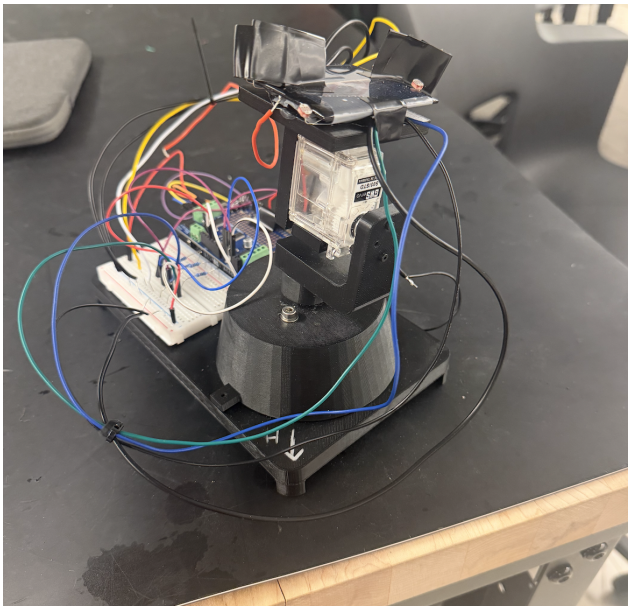


Figure 5: Dual-axis solar tracker apparatus used for experimental testing.

Table 1: Components provided in the solar tracker kit, their roles, and whether they were used in the final design.

Item	Component	Role in System	Used
1	Solar Array	Converts incident light into electrical voltage used to evaluate tracking performance and system effectiveness	Yes
2	10kΩ 10-turn potentiometer	Provides adjustable resistance for measuring angular position or calibration of rotational motion	No
3	6V DC Motor & attachments	Provides continuous vertical (altitude) motion to tilt the solar panel toward the light source	Yes
4	5V Stepper Motor	Enables precise horizontal (azimuth) rotation in controlled step increments for directional tracking	Yes
5	Arduino Uno Starter Kit	Acts as the central microcontroller, processing sensor inputs and executing control logic for motor actuation	Yes
6	Motor Shield	Interfaces motors with the Arduino, supplying required current and enabling simultaneous control of multiple motors	Yes
7	Cadmium Resistor	Detects light intensity variations at different positions to determine direction of maximum illumination	Yes
8	6-position header	Provides structured electrical connections between components on the breadboard and motor shield	Yes
9	8-position header	Provides additional connection points for organizing circuit wiring and sensor inputs	Yes
10	10-position header	Expands connectivity for power distribution and signal routing within the circuit	Yes
11	Breadboard	Allows rapid prototyping and flexible electrical connections without permanent soldering	Yes
12	Limit Switch	Mechanically restricts motion at extreme positions to prevent over-rotation and hardware damage	No
13	Mounting Hubs	Mechanically couples motors to structural components, enabling secure transmission of rotational motion	Yes

4 Results and Discussion

The solar tracking system successfully demonstrated the ability to align the solar array with the direction of maximum light intensity. During testing, the system was able to adjust its orientation across the full range of motion, responding to all tested altitude angles (15°, 45°, and 75°) and azimuth angles (45°, 90°, and 135°).

Table 2: Output voltage and tracking accuracy of the solar array for different azimuth and altitude angles within the acceptable ranges. Note: output voltage values were collected under ambient room lighting conditions and may vary slightly depending on setup and environment.

Azimuth (°)	**Altitude (°)**	**Output Voltage (V)**	**Accuracy**
45	15	2.58	Yes
45	45	2.61	Yes
45	75	2.57	Yes
90	15	2.60	Yes
90	45	2.62	Yes
90	75	2.65	Yes
135	15	2.50	No
135	45	2.61	Yes
135	75	2.64	Yes

As shown in Table 2, the system achieved correct tracking in eight out of the nine tested configurations. The tracker consistently adjusted both azimuth and altitude to follow the light source, indicating reliable dual-axis functionality. The only deviation occurred at an azimuth of 135° and an altitude of 15°, where the system did not fully align with the light source. The measured output voltage ranged from a minimum of 2.50 V to a maximum of 2.65 V. The highest voltage was observed at an azimuth of 90° and an altitude of 75°, corresponding to near-optimal alignment between the solar array and the light source. Across most configurations, voltage variation remained relatively small, suggesting that the system maintained effective alignment even as the light source position changed.

The system demonstrated smooth and continuous motion without significant mechanical interference or electrical disconnection during operation. The motors responded appropriately to sensor input, and the structure remained stable throughout testing. No major slipping or instability was observed, indicating that the mechanical design and assembly were sufficient for the intended application. The single accuracy failure at the 135° azimuth and 15° altitude condition is likely due to reduced incident light on the sensor array at shallow angles. At low altitude and large azimuth displacement, the effective light intensity detected by the photoresistors decreases, reducing the sensitivity of the tracking algorithm. This can result in incorrect directional decisions or delayed response. Future improvements could focus on increasing sensor sensitivity under low-light conditions, such as using higher-quality

photodetectors or incorporating signal amplification. Additionally, refining the control algorithm to include filtering or adaptive thresholds could improve performance near edge-case angles. Mechanical improvements, such as increasing motor torque or optimizing the mounting geometry, could also enhance responsiveness and reliability.

In addition to quantitative performance, the system demonstrated reliable mechanical and electrical operation during testing. All components remained securely connected throughout motion, and no interference was observed between moving parts or wiring. The compact design ensured that only essential components were included, contributing to a clean and efficient system layout. The tracker operated without significant slipping or instability, indicating sufficient robustness for the intended application.

5 Conclusion

This experiment evaluated the design, construction, and operation of a dual-axis solar tracking system. The system utilized a combination of photoresistor-based light sensing, an Arduino-controlled motor system, and a two-axis mechanical structure to continuously orient the solar array toward the direction of maximum light intensity. Overall, the system performed reliably, successfully tracking the light source across most tested altitude (15°, 45°, 75°) and azimuth (45°, 90°, 135°) angles. The maximum output voltage of 2.65 V was recorded at an azimuth of 90° and an altitude of 75°, corresponding to near-optimal alignment. The tracker achieved accurate positioning in eight out of nine configurations, demonstrating effective dual-axis functionality and consistent performance.

The system maintained stable operation throughout testing, with electrical and mechanical components arranged to minimize interference and ensure smooth motion. However, one performance limitation was observed at an azimuth of 135° and an altitude of 15°, where reduced light intensity at extreme angles led to tracking inaccuracy. To address the observed issues, future iterations of the system will incorporate a higher-torque DC motor to improve vertical motion stability and reduce the potential for stalling or jitter. Additionally, the current press-fit connection between the motor mount and base will be replaced with a bonded (glued) joint to improve structural rigidity and reduce unwanted vibrations during operation. Further improvements may include enhancing sensor sensitivity or refining the control algorithm to improve performance under low-light conditions. The system successfully demonstrated the feasibility of a dual-axis solar tracker capable of maintaining alignment with a moving light source under controlled conditions.

A Initial Arduino Control Code

This appendix contains the initial Arduino control program developed during the early stages of the dual-axis solar tracker design process. The purpose of the initial implementation was to establish fundamental system functionality, including light detection, motor actuation, and directional response to changes in light source position. The system utilized four light dependent resistor (LDR) sensors positioned around the solar panel assembly to detect variations in incident light intensity. The initial control strategy was designed to provide independent horizontal (azimuth) and vertical (altitude) tracking motion through comparison of sensor readings. Differences in light intensity between the left and right sensors were used to control horizontal motion, while differences between the upper and lower sensors were used to control vertical motion. This approach allowed the solar panel to rotate toward the region of highest detected illumination.

At the beginning of development, the primary objective was to verify electrical connectivity, sensor operation, and motor functionality rather than optimize tracking accuracy. The initial code implementation used threshold-based tracking logic with basic sensor calibration and direct motor actuation through the Adafruit Motor Shield. Baseline calibration values for each LDR sensor were measured during startup to compensate for differences in ambient lighting and sensor response. During preliminary testing, several tuning adjustments were required to improve overall system behavior. These included modifying motor speed settings, adjusting sensor tolerance values, and refining directional motion conventions for both the stepper motor and DC motor assemblies. Additional testing was also performed to verify correct motor direction, prevent excessive motion near balanced lighting conditions, and establish safe motion limits for both rotational axes.

```
// ============================================================
//  Dual-Axis Solar Tracker
//  Horizontal : Stepper on Motor Shield port 2 (M3/M4)
//  Vertical   : DC motor on Motor Shield port M1
//  Light      : 4 photoresistors placed at:
//               TOP, RIGHT, BOTTOM, LEFT
//
//  Change the pin assignments below to match your wiring.
// ============================================================

#include <Adafruit_MotorShield.h>

Adafruit_MotorShield AFMS = Adafruit_MotorShield();
Adafruit_StepperMotor *myStepper = AFMS.getStepper(200, 2);  // M3 + M4
Adafruit_DCMotor      *myDCMotor = AFMS.getMotor(1);         // M1

// ------------------------------------------------------------
//  SENSOR PINS
//  CHANGE THESE TO MATCH YOUR REAL WIRING
// ------------------------------------------------------------
const int PIN_TOP    = A0;
const int PIN_RIGHT  = A1;
```

```
23 const int PIN_BOTTOM = A2;
24 const int PIN_LEFT   = A3;
25
26 // ------------------------------------------------------------
27 //  DIRECTION CONVENTIONS
28 //  If motion is reversed, swap FORWARD/BACKWARD
29 // ------------------------------------------------------------
30 const uint8_t STEPPER_TOWARD_LEFT  = BACKWARD;
31 const uint8_t STEPPER_TOWARD_RIGHT = FORWARD;
32
33 const uint8_t DC_TOWARD_UP   = FORWARD;
34 const uint8_t DC_TOWARD_DOWN = BACKWARD;
35
36 // ------------------------------------------------------------
37 //  TUNING
38 // ------------------------------------------------------------
39 const int  H_TOLERANCE      = 15;
40 const int  V_TOLERANCE      = 15;
41 const int  LOOP_DELAY       = 20;
42
43 const int  STEPPER_RPM      = 10;
44 const int  STEPPER_STEP_INC = 5;
45 const long STEPPER_MAX      =  400;
46 const long STEPPER_MIN      = -400;
47
48 const uint8_t DC_SPEED      = 100;
49 const long    DC_RUN_MS     = 100;
50 const long    DC_POS_MIN    = -2500;
51 const long    DC_POS_MAX    =  2500;
52
53 const int CALIB_SAMPLES = 30;
54 const int PRINT_EVERY   = 20;
55
56 // ------------------------------------------------------------
57 //  STATE
58 // ------------------------------------------------------------
59 long stepperPos   = 0;
60 long dcPos        = 0;
61 int  printCounter = 0;
62
63 int topBase = 0, rightBase = 0, bottomBase = 0, leftBase = 0;
64
65 // ------------------------------------------------------------
66 //  HELPERS
67 // ------------------------------------------------------------
68 void runDCForMs(uint8_t direction, long ms) {
69   myDCMotor->setSpeed(DC_SPEED);
70   myDCMotor->run(direction);
71   delay(ms);
72   myDCMotor->run(RELEASE);
73 }
```

```
74
75 void calibrate() {
76   Serial.println("Calibrating... keep light steady, no flashlight.");
77   long topSum = 0, rightSum = 0, bottomSum = 0, leftSum = 0;
78
79   for (int i = 0; i < CALIB_SAMPLES; i++) {
80     topSum    += analogRead(PIN_TOP);
81     rightSum  += analogRead(PIN_RIGHT);
82     bottomSum += analogRead(PIN_BOTTOM);
83     leftSum   += analogRead(PIN_LEFT);
84     delay(50);
85   }
86
87   topBase    = topSum / CALIB_SAMPLES;
88   rightBase  = rightSum / CALIB_SAMPLES;
89   bottomBase = bottomSum / CALIB_SAMPLES;
90   leftBase   = leftSum / CALIB_SAMPLES;
91
92   Serial.print("Baselines  TOP:"); Serial.print(topBase);
93   Serial.print(" RIGHT:"); Serial.print(rightBase);
94   Serial.print(" BOTTOM:"); Serial.print(bottomBase);
95   Serial.print(" LEFT:"); Serial.println(leftBase);
96 }
97
98 // ---------------------------------------------------------------
99 //  SETUP
100 // ---------------------------------------------------------------
101 void setup() {
102   Serial.begin(9600);
103   while (!Serial);
104
105   Serial.println("Solar Tracker Starting...");
106
107   if (!AFMS.begin()) {
108     Serial.println("Motor Shield not found! Check wiring.");
109     while (1);
110   }
111
112   myStepper->setSpeed(STEPPER_RPM);
113   myDCMotor->setSpeed(0);
114   myDCMotor->run(RELEASE);
115
116   calibrate();
117   Serial.println("Ready.");
118 }
119
120 // ---------------------------------------------------------------
121 //  MAIN LOOP
122 // ---------------------------------------------------------------
123 void loop() {
124   int top    = analogRead(PIN_TOP)    - topBase;
```

```
  int right  = analogRead(PIN_RIGHT)  - rightBase;
  int bottom = analogRead(PIN_BOTTOM) - bottomBase;
  int left   = analogRead(PIN_LEFT)   - leftBase;

  // positive means more light on that side
  int vDiff = top - bottom;
  int hDiff = right - left;

  // --- Horizontal (stepper) ---
  if (abs(hDiff) > H_TOLERANCE) {
    if (hDiff > 0) {
      // more light on RIGHT -> move RIGHT
      if (stepperPos + STEPPER_STEP_INC <= STEPPER_MAX) {
        myStepper->step(STEPPER_STEP_INC, STEPPER_TOWARD_RIGHT, INTERLEAVE
   );
        stepperPos += STEPPER_STEP_INC;
      }
    } else {
      // more light on LEFT -> move LEFT
      if (stepperPos - STEPPER_STEP_INC >= STEPPER_MIN) {
        myStepper->step(STEPPER_STEP_INC, STEPPER_TOWARD_LEFT, INTERLEAVE)
   ;
        stepperPos -= STEPPER_STEP_INC;
      }
    }
  }

  // --- Vertical (DC motor) ---
  if (abs(vDiff) > V_TOLERANCE) {
    if (vDiff > 0) {
      // more light on TOP -> move UP
      if (dcPos - DC_RUN_MS >= DC_POS_MIN) {
        runDCForMs(DC_TOWARD_UP, DC_RUN_MS);
        dcPos -= DC_RUN_MS;
      }
    } else {
      // more light on BOTTOM -> move DOWN
      if (dcPos + DC_RUN_MS <= DC_POS_MAX) {
        runDCForMs(DC_TOWARD_DOWN, DC_RUN_MS);
        dcPos += DC_RUN_MS;
      }
    }
  }

  if (++printCounter >= PRINT_EVERY) {
    printCounter = 0;

    Serial.print("TOP:"); Serial.print(top);
    Serial.print(" RIGHT:"); Serial.print(right);
    Serial.print(" BOTTOM:"); Serial.print(bottom);
    Serial.print(" LEFT:"); Serial.print(left);
```

```
    Serial.print(" | hDiff:"); Serial.print(hDiff);
    Serial.print(" vDiff:"); Serial.print(vDiff);
    Serial.print(" | stepPos:"); Serial.print(stepperPos);
    Serial.print(" dcPos:"); Serial.println(dcPos);
  }

  delay(LOOP_DELAY);
}
```

Listing 1: Initial Arduino control code used during preliminary dual-axis solar tracker testing.

B Revised Arduino Control Code

This appendix presents the revised Arduino control program used for the final solar tracker testing procedure. After the initial tracking code was evaluated, the control approach was modified from continuous real-time correction to a scan-and-lock method. This revised strategy was easier to test experimentally because the tracker first searched for the brightest vertical position and then performed a horizontal scan while recording brightness values. The revised code used the four LDR sensors to calculate an average brightness value for the solar array. For vertical motion, the DC motor swept through a limited position range and stored the position that produced the highest brightness reading. After the vertical scan was complete, the motor returned to the best measured position. For horizontal motion, the stepper motor completed a continuous rotation while the Arduino monitored the brightness response and identified the step location with the greatest illumination.

Several key changes were introduced in the revised design:

- The vertical axis used a scan-and-lock method instead of continuous correction.
- The stepper motor was operated at lower speed with smaller step increments to reduce stalling.
- Sensor readings were inverted so larger values represented higher light intensity.
- The average brightness from all four LDR sensors was used to compare tracker positions.
- Serial monitor outputs were added to record sensor values, brightness trends, and motor position estimates during testing.

This revised program provided a more controlled method for validating the solar tracker response. Rather than relying only on immediate left-right and top-bottom sensor differences, the system searched through possible positions and selected the orientation with the highest measured brightness. The complete revised Arduino source code used during final testing and data collection is provided below.

```
#include <Wire.h>
#include <Adafruit_MotorShield.h>

Adafruit_MotorShield AFMS = Adafruit_MotorShield();

// DC motor on M3 = vertical
Adafruit_DCMotor *myDCMotor = AFMS.getMotor(3);

// Stepper on M1 + M2 = horizontal
Adafruit_StepperMotor *myStepper = AFMS.getStepper(200, 1);

// ------------------------------------------------------------
// SENSOR PINS
// ------------------------------------------------------------
const int PIN_TL = A0;
const int PIN_BL = A1;
const int PIN_TR = A2;
const int PIN_BR = A3;

// ------------------------------------------------------------
// DC MOTOR TUNING
// ------------------------------------------------------------
const uint8_t DC_SPEED     = 60;
const long    DC_STEP_MS   = 80;
const long    DC_POS_MAX   =  500;
const long    DC_POS_MIN   = -500;

// ------------------------------------------------------------
// STEPPER TUNING (ANTI-STALL)
// ------------------------------------------------------------
const int STEPPER_SPEED = 10;   // lower = more torque
const int STEP_SIZE = 1;        // smooth motion

// ------------------------------------------------------------
// STATE
// ------------------------------------------------------------
long dcPos = 0;
int stepperPos = 0;

// ------------------------------------------------------------
// HELPERS
// ------------------------------------------------------------
void runDCForMs(uint8_t direction, long ms) {
  myDCMotor->setSpeed(DC_SPEED);
  myDCMotor->run(direction);
  delay(ms);
  myDCMotor->run(RELEASE);
  delay(20);
}

int readSensor(int pin) {
```

```
  return 1023 - analogRead(pin); // higher = brighter
}

int readBrightness() {
  int tl = readSensor(PIN_TL);
  int tr = readSensor(PIN_TR);
  int bl = readSensor(PIN_BL);
  int br = readSensor(PIN_BR);

  return (tl + tr + bl + br) / 4;
}

void printSensorValues() {
  int tl = readSensor(PIN_TL);
  int bl = readSensor(PIN_BL);
  int tr = readSensor(PIN_TR);
  int br = readSensor(PIN_BR);

  int leftAvg = (tl + bl) / 2;
  int rightAvg = (tr + br) / 2;
  int totalAvg = (tl + bl + tr + br) / 4;

  Serial.print("TL:");
  Serial.print(tl);
  Serial.print(" BL:");
  Serial.print(bl);
  Serial.print(" TR:");
  Serial.print(tr);
  Serial.print(" BR:");
  Serial.print(br);

  Serial.print(" | Left:");
  Serial.print(leftAvg);
  Serial.print(" Right:");
  Serial.print(rightAvg);
  Serial.print(" Total:");
  Serial.println(totalAvg);
}

// ------------------------------------------------------------
// VERTICAL DC SCAN (UNCHANGED)
// ------------------------------------------------------------
void scanAndLockVertical() {
  Serial.println("=== DC Motor Vertical Scan Starting ===");
  Serial.println("Sweeping to MIN position first...");

  while (dcPos > DC_POS_MIN) {
    runDCForMs(BACKWARD, DC_STEP_MS);
    dcPos -= DC_STEP_MS;
    Serial.print("Moving to min... dcPos:");
    Serial.println(dcPos);
```

```
  }

  Serial.println("At MIN. Starting vertical scan...");
  delay(500);

  int bestBrightness = -9999;
  long bestPos = DC_POS_MIN;

  while (dcPos < DC_POS_MAX) {
    int b = readBrightness();

    Serial.print("dcPos:");
    Serial.print(dcPos);
    Serial.print(" brightness:");
    Serial.print(b);

    if (b > bestBrightness) {
      bestBrightness = b;
      bestPos = dcPos;
      Serial.print(" *** NEW BEST ***");
    }

    Serial.println();

    if (dcPos + DC_STEP_MS > DC_POS_MAX) {
      break;
    }

    runDCForMs(FORWARD, DC_STEP_MS);
    dcPos += DC_STEP_MS;
  }

  Serial.println("Vertical scan complete.");
  Serial.print("Best vertical brightness: ");
  Serial.print(bestBrightness);
  Serial.print(" at dcPos: ");
  Serial.println(bestPos);

  Serial.println("Moving vertical motor back to best position...");

  while (dcPos > bestPos) {
    runDCForMs(BACKWARD, DC_STEP_MS);
    dcPos -= DC_STEP_MS;
  }

  Serial.print("Locked vertical at dcPos:");
  Serial.println(dcPos);
}

// ------------------------------------------------------------
// HORIZONTAL STEPPER CONTINUOUS ROTATION
```

```
// ------------------------------------------------------------
void scanAndLockHorizontal() {
  Serial.println("=== Stepper Continuous Rotation Starting ===");

  int bestBrightness = -9999;
  int bestStep = 0;

  const int TOTAL_STEPS = 200; // one full revolution

  for (int i = 0; i < TOTAL_STEPS; i++) {

    int b = readBrightness();

    Serial.print("step:");
    Serial.print(i);
    Serial.print(" brightness:");
    Serial.print(b);
    Serial.print(" | ");

    printSensorValues();

    if (b > bestBrightness) {
      bestBrightness = b;
      bestStep = i;
      Serial.println("*** NEW BEST ***");
    }

    // continuous rotation
    myStepper->step(STEP_SIZE, FORWARD, DOUBLE);
    stepperPos++;

    delay(40);
  }

  Serial.println("Horizontal rotation complete.");
  Serial.print("Best brightness at step: ");
  Serial.println(bestStep);

  Serial.println("Stepper stops here (no return).");
}

// ------------------------------------------------------------
// SETUP
// ------------------------------------------------------------
void setup() {
  Serial.begin(9600);
  while (!Serial);

  if (!AFMS.begin()) {
    Serial.println("ERROR: Motor Shield not found!");
    while (1);
```

```
  }

  myDCMotor->setSpeed(0);
  myDCMotor->run(RELEASE);

  myStepper->setSpeed(STEPPER_SPEED);

  Serial.println("Vertical DC Scan -> Continuous Stepper Rotation");
  Serial.println("Starting in 3 seconds...");
  delay(3000);

  scanAndLockVertical();

  delay(1000);

  scanAndLockHorizontal();

  Serial.println("Done.");
}

// ------------------------------------------------------------
// LOOP (monitor only)
// ------------------------------------------------------------
void loop() {
  Serial.print("Holding | dcPos:");
  Serial.print(dcPos);
  Serial.print(" stepperPos:");
  Serial.print(stepperPos);
  Serial.print(" brightness:");
  Serial.print(readBrightness());
  Serial.print(" | ");

  printSensorValues();

  delay(1000);
}
```

Listing 2: Final revised Arduino control code used during experimental testing of the dual-axis solar tracker.

References

[1] Adafruit Industries, *Adafruit Motor Shield V2 Library*, GitHub Repository. Available: https://github.com/adafruit/Adafruit_Motor_Shield_V2_Library

[2] Components101, *LDR Sensor (Photoresistor) Overview and Working Principle*. Available: https://components101.com/resistors/ldr-datasheet

[3] How To Make, *How to make solar tracker using Arduino — Dual axis solar tracker*, YouTube video, 2024. Available: https://www.youtube.com/watch?v=zXX4d5eyvb8